

Results

Results I

This section reports repository facts produced by the meta-project's live introspection pass. Test counts, module inventories, pipeline-stage counts, and figure data are injected from generated artifacts. Runtime and achieved-coverage claims are intentionally omitted when no versioned benchmark or coverage artifact backs them.

Multi-Project Pipeline Execution I

The `./run.sh` interactive orchestrator can execute the public roster through the shared DAG. The table below is a structural snapshot, not a timing benchmark: counts come from repository introspection, and the coverage column states each exemplar's declared policy floor rather than an unversioned observed percentage.

Project	Effective core stages ¹	Discovered tests	Declared project floor
template_co 8		242	90%
de_project			
template_pr 8		120	90%
ose_project			
template_au 8		300	90%
toresearch_ project			

Multi-Project Pipeline Execution II

¹“Core-only” excludes LLM-tagged and other opt-in stages according to the YAML stage tags. A fresh run must be used to establish completion status or wall-clock performance for a particular machine and dependency set.

Infrastructure Test Suite I

Metric	Value
Test files	557+
Total tests	~9,557
Infrastructure coverage gate	60% configured floor
Prohibited mock-framework imports	Checked by the static no-mocks gate

Infrastructure Test Suite II

Exercises Pandoc/XeLaTeX paths, steganography hashing, telemetry, YAML-driven pipeline DAG resolution, HTTPS-bound optional suites (`pytest-httpserver`), and local Ollama-gated subsets.

Infrastructure Module Inventory I

The introspection module (`template_template.introspection`) emits the authoritative table below—every row reflects `discover_infrastructure_modules(REPO_ROOT)`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	10			build_autoresearch_plan, readiness validation CLI
benchmark	4			Template harness scoring + comparative gates
config	0			Repository defaults + hardened templates
core	120			get_logger, load_config, TemplateError
docker	0			Containerisation scaffolding
doctor	14			Checkout diagnose/fix/undo repairs
documentation	14			FigureManager, generate_rate_glossary
fonds	6			—
llm	55			Ollama helpers, sanitization, review + translation pipelines
logrotate.d	0			Rotation snippets (documentation-first)
methods	5			build_methods_orchestration_plan, methods-stage contracts + validation

Infrastructure Module Inventory II

orchestration	12	PipelineRunner, entry point for
project	41	./run.sh discover_projects, workspace
prose	9	management Markdown readability + prose tooling
provenance	7	—
publishing	81	Zenodo, executable bundle, archival targets
reference	16	BibTeX models, parsers, converters
rendering	65	PDF/HTML/slide rendering, Pandoc filters
reporting	57	Coverage parsers, dashboards, executive artefacts
research	3	—
rules	6	—
scientific	4	check_numerical_sta bility, benchmark_f unction
search	62	infrastructure.sear ch.literature clients + cache
sia	10	Self-Improving-AI loop: task validation, harness, metric capture
skills	8	discover_skills, SKILL manifest regeneration

Infrastructure Module Inventory III

steganography	13	Watermark overlays + hash manifests
tools	6	—
validation	80	PDF + Markdown + integrity CLIs

Infrastructure Module Inventory IV

All 28 enumerated subdirectories carry Tier-1/README.md and Tier-2/AGENTS.md coverage wherever the Documentation Duality standard applies; subsets ship Tier-3 SKILL.md descriptors for MCP routing (infrastructure/skills manifest generation).

Agentic Skill Documentation Coverage I

Layer	Role
System prompts	Root CLAUDE.md, README policy
Structural	AGENTS.md directories
Skills	Optional SKILL.md manifests + generated .cursor/skill_manifest.json
PAI capsule	Repository level PAI.md narratives

Agentic Skill Documentation Coverage II

404+ Markdown shards under docs/ capture operational knowledge without duplicating auto-generated inventories.

DAG Reference (Declarative Executor) I

Stages below mirror `pipeline.yaml` (executor-topological order—not strict numeric script filenames). Scripts live under `scripts/`.

Name	Typical script / method	Responsibility	Failure semantics
Clean Output Directories	<code>_run_clean_outputs</code>	Deletes stale output/ trees	Blocking
Environment Setup	<code>scripts/pipeline/stage_00_setup.py</code>	Validates tooling, PYTHONPATH scaffolding	Blocking
Infrastructure Tests	<code>scripts/pipeline/stage_01_test.py -infra-only</code>	Infra pytest + coverage gates	Tunable thresholds
Project Tests	<code>scripts/pipeline/stage_01_test.py -project-only</code>	Project pytest + coverage gates	Zero failures default
Project Analysis	<code>scripts/pipeline/stage_02_analysis.py</code>	Executes projects/<name>/scripts/*.py	Blocking
PDF Rendering	<code>scripts/pipeline/stage_03_render.py</code>	Pandoc → XeLaTeX manuscripts	Blocking
Output Validation	<code>scripts/pipeline/stage_04_validate.py</code>	Structural PDF/markdown probes	Blocking / warnings
LLM Scientific Review	<code>scripts/pipeline/stage_06_llm_review.py --reviews-only</code>	Local Ollama reviews	Skippable / exit 2 tolerated
LLM Translations	<code>scripts/pipeline/stage_06_llm_review.py --translations-only</code>	Optional translations	Skippable
Copy Outputs	<code>scripts/pipeline/stage_05_copy.py</code>	Mirrors deliverables → output/<project>/	Soft-fail surfaced in logs

DAG Reference (Declarative Executor) II

`scripts/pipeline/stage_07_executive_report.py` is **multi-project orchestration glue** invoked after iterating active projects—not a tenth DAG node for single-repo runs (`execute_pipeline.py`).

Steganographic Performance Boundary I

The steganography path exposes metadata injection, SHA-256 hashing, overlay generation, and optional QR-code insertion as separately testable operations. This revision makes no latency claim because the project does not currently generate a versioned benchmark artifact recording hardware, input PDF, warm-up policy, repetitions, and dispersion. A future performance table should be compiled from such an artifact rather than copied from an ad-hoc workstation run.

Self-Referential Analysis I

Rendered via `projects/templates/template_template` (`generate_manuscript_metrics.py` → injected tokens such as 28).
Architecture figures stem from `template_template.architecture_viz`—font sizes constrained by §QA.

Self-Differential Analysis II

Two-Layer Architecture

Layer 1: Infrastructure (generic · reusable · cross-project)

Layer 2: Projects (domain-specific · customizable · isolated)

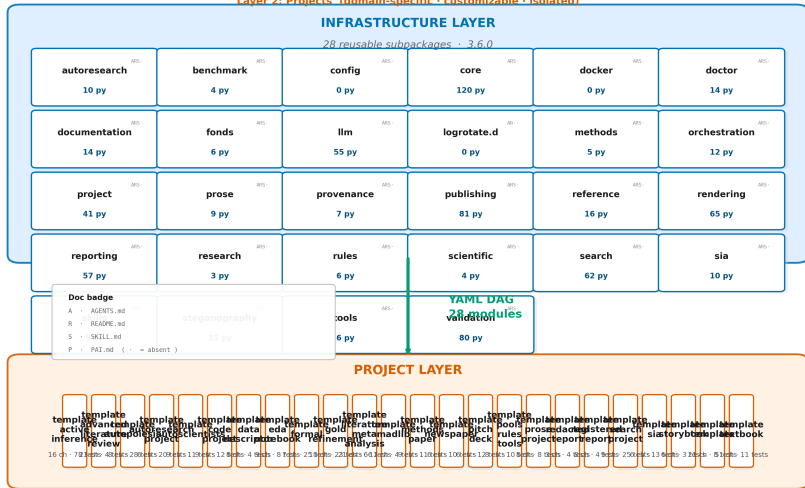


Figure 1. Live rendering of the Two-Layer Architecture from repository introspection: the infrastructure layer (top) holds the 28 reusable subpackages, each annotated with its Python file count and a four slot documentation badge—A AGENTS.md, B

Comparative Feature Analysis I

Figure 4 summarizes the Appendix F capability matrix.

Comparative Feature Analysis II

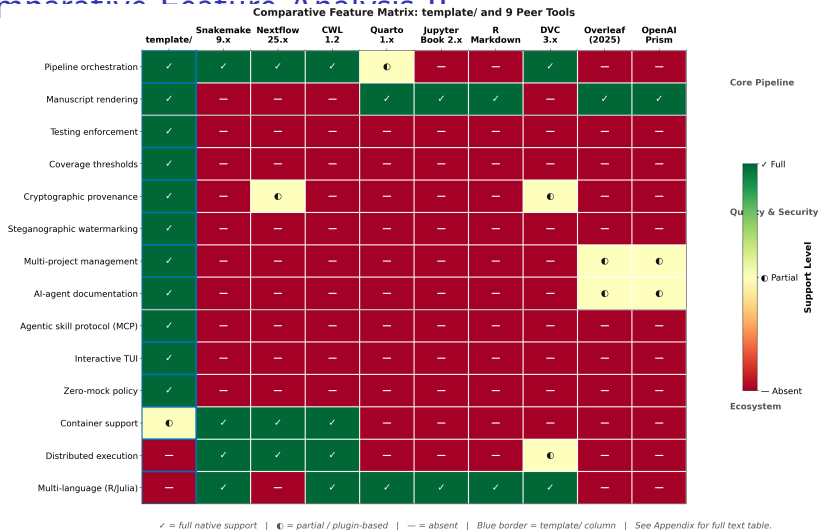


Figure 4. 14 × 10 heatmap annotated in appendix text—green full native capability, yellow partial / plugin-hosted, red — unavailable. Rows group *Core Pipeline*, *Quality & Security*, then *Ecosystem*.

Test Quality Metrics I

- ▶ **Zero mocks:** repository policy bans `unittest.mock` / patching frameworks in tests.
- ▶ **Filesystem + subprocess realism:** ephemeral directories + actual CLI binaries.
- ▶ **HTTP realism:** infra suites favour `pytest-httpserver`; literature tests hit recorded fixtures.
- ▶ `template_code_project` focuses on numerical reproducibility assertions.
- ▶ `template_autoresearch_project` exercises the AutoResearch readiness planner (`infrastructure/autoresearch/`). `template_search_project` exercises literature-search workflows.